

User Manual

for S32K1_S32M24X DIO Driver

Document Number: UM2DIOASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	6
3.4 Deviations from Requirements	7
3.5 Driver Limitations	8
3.6 Driver usage and configuration tips	8
3.7 Runtime errors	10
3.8 Symbolic Names Disclaimer	11
3.9 Usage in non AUTOSAR context	11
4 Configuration Parameters	13
4.1 Dio	14
4.1.1 Module Dio	15
4.1.2 Container DioConfig	15
4.1.3 Container DioPort	15
4.1.4 Parameter DioPortId	16
4.1.5 Reference DioPortEcucPartitionRef	16
4.1.6 Container DioChannel	17
4.1.7 Parameter DioChannelId	17
4.1.8 Reference DioChannelEcucPartitionRef	18
4.1.9 Container DioChannelGroup	18
4.1.10 Parameter DioChannelGroupIdentification	18
4.1.11 Parameter DioPortBitNumber	19
4.1.12 Parameter DioPortOffset	19
4.1.13 Parameter DioPortMask	20
4.1.14 Reference DioChannelGroupEcucPartitionRef	20
4.1.15 Container DioGeneral	21
4.1.16 Parameter DioDevErrorDetect	21
4.1.17 Parameter GPIODioIPDevErrorDetect	21
4.1.18 Parameter DioVersionInfoApi	22
4.1.19 Parameter DioReversePortBits	22
4.1.20 Parameter DioFlipChannelApi	23
4.1.21 Parameter DioReadZeroForUndefinedPortPins	23
4.1.22 Parameter DioMaskedWritePortApi	24
4.1.23 Parameter DioEnableUserModeSupport	24
4.1.24 Parameter DioMultiPartitionSupport	25
4.1.25 Reference DioEcucPartitionRef	25
4.1.26 Container CommonPublishedInformation	26
4.1.27 Parameter ArReleaseMajorVersion	26
4.1.28 Parameter ArReleaseMinorVersion	26
4.1.29 Parameter ArReleaseRevisionVersion	27
4.1.30 Parameter ModuleId	27
4.1.31 Parameter SwMajorVersion	28
4.1.32 Parameter SwMinorVersion	28

4.1.33 Parameter SwPatchVersion	28
4.1.34 Parameter VendorApiInfix	29
4.1.35 Parameter VendorId	29
4.2 GPIO_DIO	30
4.2.1 Module Gpio_Dio	30
5 Module Index	31
5.1 Software Specification	31
6 Module Documentation	32
6.1 Dio HLD	32
6.1.1 Detailed Description	32
6.1.2 Macro Definition Documentation	33
6.1.2.1 DIO_E_PARAM_CONFIG	33
6.1.2.2 DIO_E_PARAM_INVALID_CHANNEL_ID	33
6.1.2.3 DIO_E_PARAM_INVALID_PORT_ID	33
6.1.2.4 DIO_E_PARAM_INVALID_GROUP_ID	33
6.1.2.5 DIO_E_PARAM_POINTER	34
6.1.2.6 DIO_E_PARAM_LEVEL	34
6.1.2.7 DIO_READCHANNEL_ID	34
6.1.2.8 DIO_WRITECHANNEL_ID	34
6.1.2.9 DIO_FLIPCHANNEL_ID	34
6.1.2.10 DIO_READPORT_ID	35
6.1.2.11 DIO_WRITEPORT_ID	35
6.1.2.12 DIO_READCHANNELGROUP_ID	35
6.1.2.13 DIO_WRITECHANNELGROUP_ID	35
6.1.2.14 DIO_GETVERSIONINFO_ID	35
6.1.2.15 DIO_MASKEDWRITEPORT_ID	36
6.1.2.16 DIO_INSTANCE_ID	36
6.1.3 Function Reference	36
6.1.3.1 Dio_ReadChannel()	36
6.1.3.2 Dio_WriteChannel()	36
6.2 Dio IPL	37
6.2.1 Detailed Description	37
6.2.2 Types Reference	37
6.2.2.1 Gpio_Dio_Ip_PinsChannelType	37
6.2.2.2 Gpio_Dio_Ip_PinsLevelType	38
6.2.3 Function Reference	38
6.2.3.1 Gpio_Dio_Ip_WritePin()	38
6.2.3.2 Gpio_Dio_Ip_WritePins()	38
6.2.3.3 Gpio_Dio_Ip_GetPinsOutput()	39
6.2.3.4 Gpio_Dio_Ip_SetPins()	39
6.2.3.5 Gpio_Dio_Ip_ClearPins()	39
6.2.3.6 Gpio_Dio_Ip_TogglePins()	40
6.2.3.7 Gpio_Dio_Ip_ReadPins()	40
6.2.3.8 Gpio_Dio_Ip_ReadPin()	41



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor AUTOSAR DIO for S32K1XX & S32M24X. AUTOSAR DIO driver configuration parameters and deviations from the specification are described in Driver chapter of this document. AUTOSAR DIO driver requirements and APIs are described in the AUTOSAR DIO driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64

- s32k146_lqfp64
- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of Driver	AUTOSAR Release R21-11
2	S32K1xx Reference Manual	Rev.14.1, 01/2024
3	S32M24x Reference Manual	Rev. 5, 12/2024
4	S32K1xx Data Sheet	Rev. 14, 08/2021
5	S32M2xx Data Sheet	Rev. 7, 12/2024
6	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
7	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
8	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
9	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
10	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
11	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
12	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
13	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
14	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See [Table Reference List](#)).

3.2 Driver Design Summary

The DIO Driver provides services for reading and writing to/from:

- DIO Channels (Pins)
- DIO Ports
- DIO Channel Groups

The behaviour of those services is synchronous. This module works on pins and ports which are configured by the PORT driver for this purpose. For this reason, there is no configuration and initialization of this port structure in the DIO Driver.

3.3 Hardware Resources

The hardware configured by the Dio driver is GPIO. The channel to microcontroller pin mapping can be done by using the corresponding S32K1XX or S32M24X IO Muxing documentations.

Value of actual channel is identified by formula:

Channel = DioChannelId + DioPortId*32

Where:

- DioPortId is the numeric identifier of the DIO port. Symbolic names will be generated for each port pin id for the pins which being used for configuration.
 - PortA: DioPortId = 0
 - PortB: DioPortId = 1
 - PortC: DioPortId = 2
 - PortD: DioPortId = 3
 - PortE: DioPortId = 4
- DioChannelId is selected channel in the port what is selected by choosing the value of DioPortId. The maximum channel in 1 port is 32, so the range of DioChannelId is: 0-31

Eg. Channel PTB5 can be found in the S32M24x_IOMUX.xlsx file, which is connected to PortPin Pcr 37. In order to use PTB5 in the Dio driver, the corresponding configuration values is DioChannelId = 5 and DioPortId = 1 (Port B channel 5).

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR DIO Driver software specification in some places. The table identifies the AUTOSAR requirements that are not fully implemented, not implemented or out of scope for the DIO Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, not implemented or out of scope for the driver.

Requirement	Status	Description	Notes
SWS_Dio_00083	N/S	If the microcontroller supports the direct readback of a pin value, the Dio module's read functions shall provide the real pin level, when they are used on a channel which is configured as an output channel	This works only if the output channel is configured as input, too. This requirement is rejected and replaced by CPR_RTD_00001.dio
SWS_Dio_00084	N/S	If the microcontroller does not support the direct read-back of a pin value, the Dio module's read functions shall provide the value of the output register, when they are used on a channel which is configured as an output channel.	The read functions will only read the input registers, regardless of the channel configuration. This requirement is rejected and replaced by CPR_RTD_00001.dio
SWS_Dio_00104	N/S	When reading a port which is smaller than the Dio_PortType using the Dio_ReadPort function (see [SWS_Dio_00103]), the function shall set the bits corresponding to undefined port pins to 0. Furthermore, the requirements SWS_Dio_00005, SWS_Dio_00118 and SWS_Dio_00026 are applicable to the Dio_ReadPort function.	Requirement not applicable. Dio_ReadPort function always reads a port of exactly the size defined by Dio_PortType. It cannot read a port smaller than the size of the Dio_PortType.

Requirement	Status	Description	Notes
SWS_Dio_00105	N/S	When writing a port which is smaller than the Dio_PortType using the Dio_WritePort function (see [SWS_Dio_00103]), the function shall ignore the MSB.	Requirement not applicable. Dio_WritePort function always writes a port of exactly the size defined by Dio_PortType. It cannot write a port smaller than the size of the Dio_PortType.
SWS_Dio_00195	N/S	These requirements are not applicable to this specification.	This is not a requirement.
SWS_Dio_00204	N/S	When writing a port which is smaller than the Dio_PortLevelType using the Dio_MaskedWritePort function (see [SWS_Dio_00103]), the function shall ignore the MSB.	Dio_MaskedWritePort function always reads a port of exactly the size defined by Dio_PortLevelType. It cannot read a port smaller than the size of the Dio_PortLevelType.
SWS_Dio_00210	N/S	If DioEcucPartitionRef references one or more ECUC partitions, DioPortEcucPartitionRef shall have a multiplicity of greater than zero and reference one or several of these ECUC partitions as well.	[17-Aug-22] Requirement is duplicated with SWS_Dio_00209.

3.5 Driver Limitations

None

3.6 Driver usage and configuration tips

The Dio driver APIs work with channels, ports and channel groups.

Dio channels

A channel is represented by a microcontroller hardware pin. In order to be able to use the Dio channel APIs ([Dio_ReadChannel\(\)](#), [Dio_WriteChannel\(\)](#) and [Dio_FlipChannel\(\)](#)) for a specific pin, there are a couple steps to be done:

- Open the platform reference manual or the IoMuxing Excel attached to it
- Identify the microcontroller pin you want to use (eg. PTE[5])
- Go to DioPort container inside the Dio plugin and add a new port
- Click on the Dio Port Id attribute and observe the content of the Description field
- Take the numeric identifier of the port containing the pin you want to use (eg. 4 corresponding to port E for PTE[5]) and set the Dio Port Id to this value
- Go to the DioChannel container inside the DioPort container and add a new channel
- Take the numeric identifier of the pin inside the port for the hardware pin you want to use (eg. 5 for PTE[5]) and set the Dio Channel Id attribute to this value
- Generate the code
- Go to Dio_Cfg.h file and look inside the 'DEFINES AND MACROS' section of the file for a define that represents the symbolic name of the Dio Channel (eg. DioConf_DioChannel_DioChannel_0)
- Always use this define as ChannelId parameter when calling Dio APIs related to channels ([Dio_ReadChannel\(\)](#), [Dio_WriteChannel\(\)](#) and [Dio_FlipChannel\(\)](#))

Dio Ports

A port represents several DIO channels that are grouped by hardware (typically controlled by one hardware register). In order to be able to use the Dio port APIs (`Dio_ReadPort()`, `Dio_WritePort()` and `Dio_MaskedWritePort()`) for a specific port, there are a couple steps to be done:

- Open the platform reference manual or the IoMuxing Excel attached to it
- Identify the microcontroller port you want to use (eg. PTE)
- Go to DioPort container inside the Dio plugin and add a new port
- Click on the Dio Port Id attribute and observe the content of the Description field
- Take the numeric identifier of the port you want to use (eg. 4 corresponding to port E) and set the Dio Port Id to this value
- Generate the code
- Go to `Dio_Cfg.h` file and look inside the 'DEFINES AND MACROS' section of the file for a define that represents the symbolic name of the Dio port (eg. `DioConf_DioPort_DioPort_0`)
- Always use this define as PortId parameter when calling Dio APIs related to ports (`Dio_ReadPort()`, `Dio_WritePort()` and `Dio_MaskedWritePort()`)

Dio channel groups

A Dio channel group consists of several adjoining Dio channels that belong to one Dio port. In order to be able to use the Dio channel group APIs (`Dio_ReadChannelGroup()`, `Dio_WriteChannelGroup()`), there are a couple steps to be done:

- Open the platform reference manual or the IoMuxing Excel attached to it
- Identify the microcontroller pins you want to use (eg. PTE[5], PTE[6], PTE[7])
- Go to DioPort container inside the Dio plugin and add a new port
- Click on the Dio Port Id attribute and observe the content of the Description field
- Take the numeric identifier of the port containing the pin you want to use (eg. 4 corresponding to port E for PTE[5], PTE[6], PTE[7]) and set the Dio Port Id to this value
- Go to the DioChannelGroup container inside the DioPort container and add a new channel group
- Configure the channel group. The information that is needed by the driver is the one in the 'Dio Port Mask' attribute. There is no need to write that information directly, the attributes 'Dio Port Bit Number' and 'Dio Port Offset' are here to help. Just fill them with the number of continuous channels that create the channel group and with the position of the channel group in the port, counted from the least significant bit and hit the 'Calculate value' button on the right side of the 'Dio Port Mask' attribute
- Generate the code
- Go to `Dio_Cfg.h` file and look inside the 'DEFINES AND MACROS' section of the file for a define that represents the symbolic name of the Dio Channel Group (eg. `DioConf_DioChannelGroup_DioChannelGroup_0`)
- Always use this define as ChannelGroupIdPtr parameter when calling Dio APIs related to channel groups (`Dio_ReadChannelGroup()`, `Dio_WriteChannelGroup()`)

Autosar extension functionality

- 1. Reverse bits in ports. This option is configurable on/off per entire driver, using the checkbox 'Dio Reverse Port Bits' in DioGeneral container. It affects the functionality of the following APIs working with Dio ports:

Dio_ReadPort(), Dio_WritePort(), Dio_ReadChannelGroup() and Dio_WriteChannelGroup(). If the 'Dio Reverse Port Bits' box is checked, the bits written to ports by the 4 functions above will be reversed. For example, writing 3 to a port with checkbox disabled will set pins 0 and 1 while writing 3 to a port with checkbox enabled will set pins 14 and 15 if the port has 16 bits width or pins 30 and 31 if the port has 32 bits width.

- 2. Read zero for undefined port pins. This option is configurable on/off per entire driver, using the checkbox 'Dio Read Zero For Undefined Port Pins' in DioGeneral container. It affects the functionality of the Dio_↔ReadPort() API. It is possible for a given microcontroller port to not have all pins physically implemented. Checking this option will ensure that all not implemented pins in a port read will be read as 0 logic when API Dio_ReadPort() is called for that port.
- 3. Support to run driver's code from User Mode. This option is configurable on/off per entire driver, using the checkbox 'Enable Dio User Mode Support' in DioGeneral container. When this parameter is enabled, the Dio module will adapt to run from user mode so that the registers under protection can be accessed from user mode. For more information, please see the IM chapter 'User Mode Support'.
- 4. API to write a port using mask. In DioGeneral container there is an attribute called 'Dio Masked Write Port Api'. If the attribute is checked, the Dio driver code will include one extra API for writing the value of a port, called Dio_MaskedWritePort(). Compared with the Dio_WritePort() API, this function has one extra parameter called 'Mask', which has the size of the port width. When using this API, only the port channels having the corresponding bits in the 'Mask' set to 1 will be set to the value of the corresponding bits in the 'Level' parameter.

3.7 Runtime errors

The driver does not trigger any DEM runtime error, but triggers the runtime DET errors listed in the table below:

Function	Error Param	Condition triggering the error
Dio_ValidateChannelLevel	DIO_E_PARAM_LEVEL	Invalid dio channel level
Dio_GetVersionInfo	DIO_E_PARAM_POINTER	The "VersionInfo" param is NULL pointer
Dio_ValidateChannelForWrite	DIO_E_PARAM_INVALID_↔CHANNEL_ID	Invalid dio channel id
Dio_ValidateChannelForRead	DIO_E_PARAM_INVALID_↔CHANNEL_ID	Invalid dio channel id
Dio_ValidatePortForWrite	DIO_E_PARAM_INVALID_↔PORT_ID	Invalid port id
Dio_ValidatePortForRead	DIO_E_PARAM_INVALID_↔PORT_ID	Invalid port id
Dio_ValidateChannelGroupFor↔Read	DIO_E_PARAM_INVALID_↔GROUP_ID	Invalid channel group
Dio_ValidateChannelGroupFor↔Write	DIO_E_PARAM_INVALID_↔GROUP_ID	Invalid channel group
Dio_WriteChannelGroup	DIO_E_PARAM_CONFIG	Invalid config parameter
Dio_ReadChannelGroup	DIO_E_PARAM_CONFIG	Invalid config parameter
Dio_ValidateChannelForRead	DIO_E_PARAM_CONFIG	Invalid config parameter
Dio_ValidateChannelForWrite	DIO_E_PARAM_CONFIG	Invalid config parameter
Dio_ValidatePortForRead	DIO_E_PARAM_CONFIG	Invalid config parameter
Dio_ValidatePortForWrite	DIO_E_PARAM_CONFIG	Invalid config parameter

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

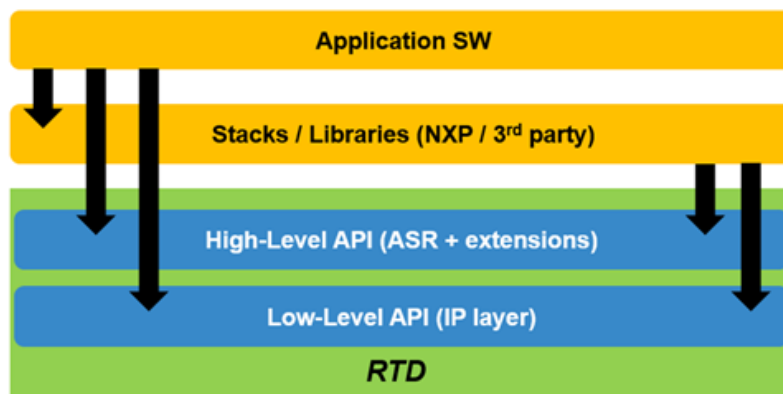


Figure 3.1 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized
 2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
 3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
 4. Initialization of specific drivers
 5. Runtime functionality
 6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity

considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

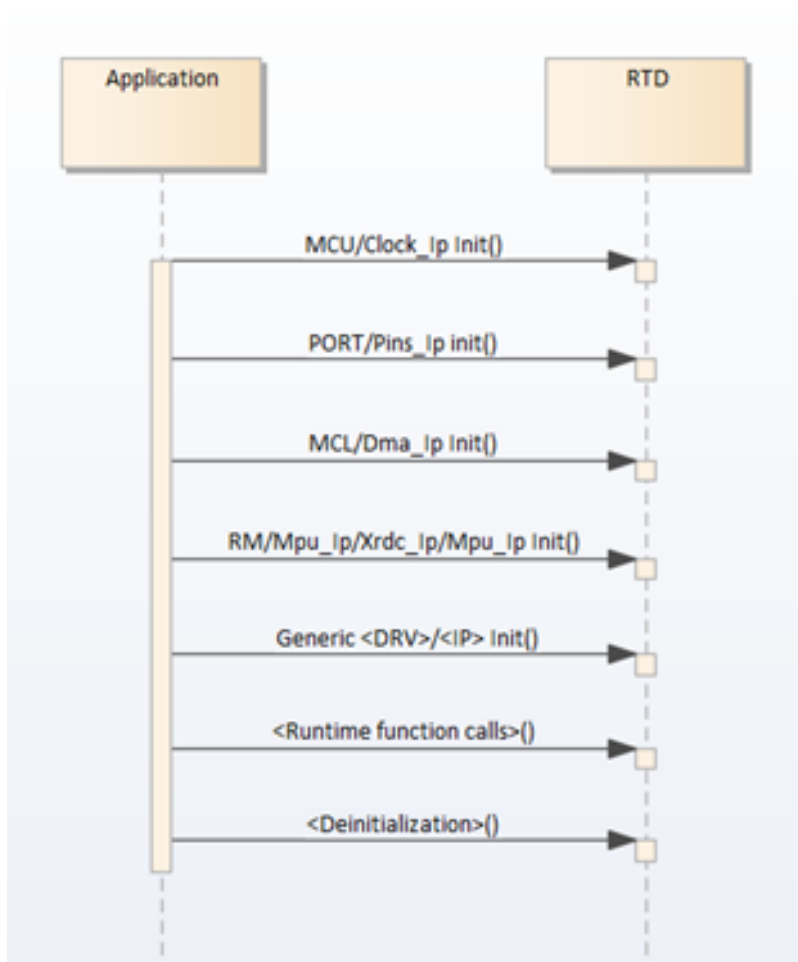


Figure 3.2 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

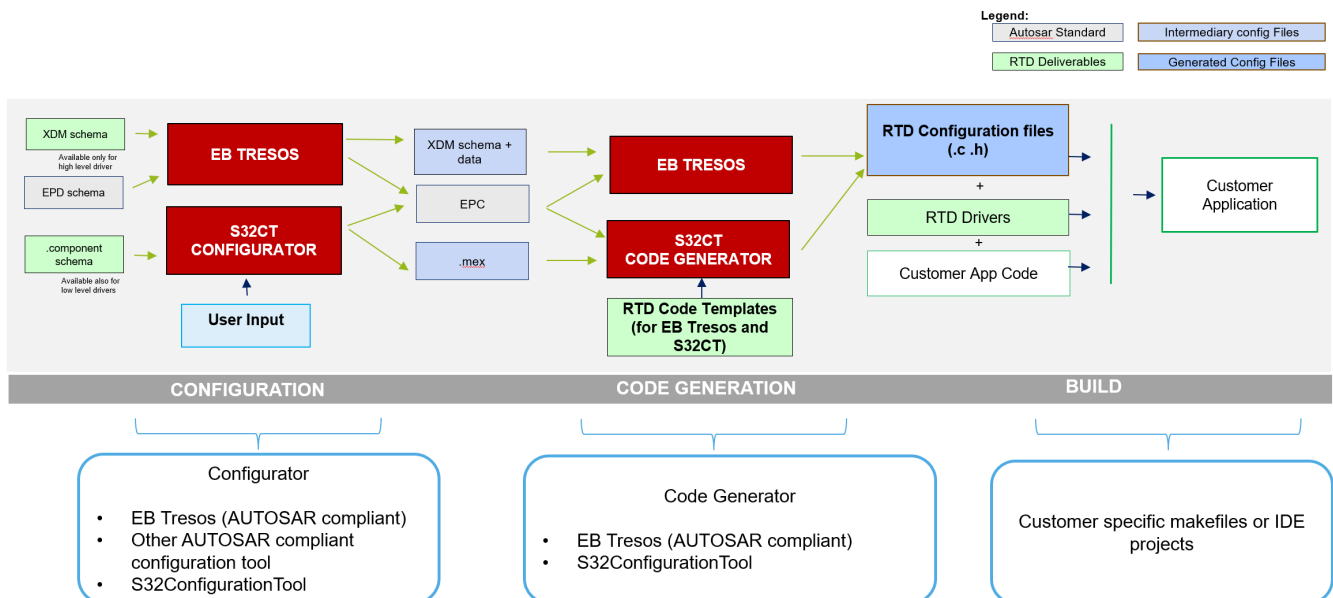


Figure 4.1 Configuration generation flow

- [Dio](#)
- [GPIO_DIO](#)

4.1 Dio

- Module [Dio](#)
 - Container [DioConfig](#)
 - * Container [DioPort](#)
 - Parameter [DioPortId](#)
 - Reference [DioPortEcucPartitionRef](#)
 - Container [DioChannel](#)
 - Parameter [DioChannelId](#)
 - Reference [DioChannelEcucPartitionRef](#)
 - Container [DioChannelGroup](#)
 - Parameter [DioChannelGroupIdentification](#)
 - Parameter [DioPortBitNumber](#)
 - Parameter [DioPortOffset](#)
 - Parameter [DioPortMask](#)
 - Reference [DioChannelGroupEcucPartitionRef](#)
 - Container [DioGeneral](#)
 - * Parameter [DioDevErrorDetect](#)
 - * Parameter [GPIODioIPDevErrorDetect](#)
 - * Parameter [DioVersionInfoApi](#)
 - * Parameter [DioReversePortBits](#)
 - * Parameter [DioFlipChannelApi](#)
 - * Parameter [DioReadZeroForUndefinedPortPins](#)
 - * Parameter [DioMaskedWritePortApi](#)
 - * Parameter [DioEnableUserModeSupport](#)
 - * Parameter [DioMultiPartitionSupport](#)
 - * Reference [DioEcucPartitionRef](#)
 - Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1.1 Module Dio

Configuration of the Dio (Digital IO) module.

Included containers:

- [DioConfig](#)
- [DioGeneral](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	false
supportedConfigVariants	VARIANT-LINK-TIME, VARIANT-PRE-COMPILE

4.1.2 Container DioConfig

This container contains the configuration parameters and sub containers of the AUTOSAR DIO module. This container is a MultipleConfigurationContainer, i.e. this container and its sub-containers exist once per configuration set.

Included subcontainers:

- [DioPort](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Container DioPort

Configuration of individual DIO ports, consisting of channels and possible channel groups.

The single DIO channel levels inside a DIO port represent a bit in the DIO port value. A channel group is a formal logical combination of several adjoining DIO channels within a DIO port. The configuration process for Dio module shall provide symbolic names for each configured DIO channel, port and group.

Included subcontainers:

- [DioChannel](#)
- [DioChannelGroup](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite

Property	Value
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.4 Parameter DioPortId

Numeric identifier of the DIO port. Symbolic names will be generated for each port pin id for the pins which being used for configuration.

NOTE: Use the following values to configure different ports.

PortA=0

PortB=1

PortC=2

PortD=3

PortE=4

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4
min	0

4.1.5 Reference DioPortEcucPartitionRef

Maps the Dio Port to zero a multiple ECUC partitions. The ECUC partitions referenced are a subset of the ECUC partitions where the Dio driver is mapped to.

EN: Tags: atp.Status=draft

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.6 Container DioChannel

Configuration of an individual DIO channel. Symbolic names will be generated for each channel.

A general purpose digital IO pin represents a DIO channel which will be having value either STD_HIGH or STD_LOW.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.7 Parameter DioChannelId

Channel Id of the DIO channel. This value will be assigned to the symbolic names.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	31
min	0

4.1.8 Reference DioChannelEcucPartitionRef

EN: Maps the Dio Channel to zero a multiple ECUC partitions. The ECUC partitions referenced are a subset of the ECUC partitions where the related Dio port is mapped to.

EN: Tags: atp.Status=draft

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.9 Container DioChannelGroup

A channel group represents several adjoining DIO channels represented by a logical group.

This container definition does not explicitly define a symbolic name parameter, but symbolic names will be generated for each channel group. Each group provides a structure with parameters port, offset and mask.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	255
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.10 Parameter DioChannelGroupIdentification

A DIO channel group is identified in DIO APIs by a pointer to a data structure of type Dio_ChannelGroupType. This data structure contains the channel group information. This parameter contains the code fragment that has to be inserted in the API call of the calling module to get the address

of the variable in memory which holds the channel group information, a string value should be given for this parameter. Symbolic names will be generated for

each DioChannelGroup, which will be assigned with address of this string inorder to point to the structure parameters. Example: OutputGroup

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	DioChannelGroup

4.1.11 Parameter DioPortBitNumber

This is the number of continuous channels that create a channel group

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	1
max	32
min	1

4.1.12 Parameter DioPortOffset

The position of the Channel Group on the port, counted from the least significant bit. This value can be derived from DioPortMask.

calculationFormula = Position of the first bit of DioPortMask which is set to 1 counted from least significant bit

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	31
min	0

4.1.13 Parameter DioPortMask

This shall be the mask which defines the positions of the channel group.

This field holds a decimal value that, when converted into binary, represents a bitmask for the channels in the group(e.g a value of 14 = 0x0E selects channels 1, 2 and 3).

The binary value of the mask should have a single continuous group of 1 bits.

The data type depends on the port width.

To display the correct result you must click on the "calculator" icon near the Dio Port Mask field

Calculation formula: DioPortMask = DioPortBitNumber « DioPortOffset

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.1.14 Reference DioChannelGroupEcucPartitionRef

EN: Maps the Dio Channel Group to zero a multiple ECUC partitions. The ECUC partitions referenced are a subset of the ECUC partitions where the related Dio port is mapped to.

EN: Tags: atp.Status=draft

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false

Property	Value
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.15 Container DioGeneral

General DIO module configuration parameters.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.16 Parameter DioDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

True: Enabled.

False: Disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.17 Parameter GPIODioIPDevErrorDetect

Enables and Disables DevAssert checks in IP code.

Configuration Parameters

True: Enabled.

False: Disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.18 Parameter DioVersionInfoApi

Adds / removes the service Dio_GetVersionInfo() from the code.

True - Dio_GetVersionInfo() API is enabled.

False - Dio_GetVersionInfo() API is disabled (it cannot be used).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.19 Parameter DioReversePortBits

If this box is checked, the bits written to defined ports will be reversed,

meaning that writing 3 to a port with checkbox disabled will set pins 0 and 1 of the port

while writing 3 to a port with checkbox enabled will set pins 30 and 31 of the port.

This functionality is an AutoSAR extension.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.20 Parameter DioFlipChannelApi

Adds / removes the service `Dio_FlipChannel()` from the code.

True - `Dio_FlipChannel()` API is enabled.

False - `Dio_FlipChannel()` API is disabled (it cannot be used).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.21 Parameter DioReadZeroForUndefinedPortPins

Defines whether the `Dio_ReadPort()` function includes the capability to read the undefined port pins as 0.

True - Enables the `Dio_ReadPort()` functionality to read the undefined port pins as 0.

False - Disables the `Dio_ReadPort()` functionality to read the undefined port pins as 0

(Supports the normal functionality with `Dio_ReadPort()`).

This functionality is an AutoSAR extension.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.22 Parameter DioMaskedWritePortApi

Defines whether the driver function Dio_MaskedWritePort() will be included at compile time or excluded.

This API is an AutoSAR extension.

True - Dio_MaskedWritePort() API enabled.

False - Dio_MaskedWritePort() API disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.23 Parameter DioEnableUserModeSupport

This parameter is added in Dio configuration in order to keep a consistent design over the entire set of RTD drivers.

It cannot be configured by the user and is always set to 'false'.

There are no registers used by the driver which require special measures in order to be accessed from user mode, so Dio driver can be run from either user or supervisor mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.24 Parameter DioMultiPartitionSupport

This parameter globally enables the possibility to support MultiPartition. If this parameter is enabled, at least one EcucPartition needs to be defined (in all variants).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.25 Reference DioEcucPartitionRef

Maps the Dio driver to zero a multiple ECUC partitions to make the modules API available in this partition.

Tags: atp.Status=draft

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.26 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.27 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.28 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION

Property	Value
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.29 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.30 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	120
max	120
min	120

4.1.31 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.32 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.33 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.34 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

`<ModuleName>_<VendorId>_<VendorApiInfix>`.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name `Can_Write` defined in the SWS will translate to `Can_123_v11r456Write`.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.1.35 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.2 GPIO_DIO

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Gpio_Dio](#)
 - Container [DioGeneral](#)
 - * Parameter [GPIODioIPDevErrorDetect](#)

4.2.1 Module Gpio_Dio

Dio Driver

Included containers:

- [DioGeneral](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	false
supportedConfigVariants	VARIANT-LINK-TIME;VARIANT-PRE-COMPILE



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Dio HLD	32
Dio IPL	37

Chapter 6

Module Documentation

6.1 Dio HLD

6.1.1 Detailed Description

Macros

- `#define DIO_E_PARAM_CONFIG`
The DIO module is not properly configured.
- `#define DIO_E_PARAM_INVALID_CHANNEL_ID`
Invalid channel name requested.
- `#define DIO_E_PARAM_INVALID_PORT_ID`
Invalid port name requested.
- `#define DIO_E_PARAM_INVALID_GROUP_ID`
Invalid ChannelGroup id passed.
- `#define DIO_E_PARAM_POINTER`
API service called with a NULL pointer.
- `#define DIO_E_PARAM_LEVEL`
API service called with invalid channel level value.
- `#define DIO_READCHANNEL_ID`
API service ID for `Dio_ReadChannel()` function.
- `#define DIO_WRITECHANNEL_ID`
API service ID for `Dio_WriteChannel()` function.
- `#define DIO_FLIPCHANNEL_ID`
API service ID for `Dio_FlipChannel()` function.
- `#define DIO_READPORT_ID`
API service ID for `Dio_ReadPort()` function.
- `#define DIO_WRITEPORT_ID`
API service ID for `Dio_WritePort()` function.
- `#define DIO_READCHANNELGROUP_ID`
API service ID for `Dio_ReadChannel()` Group function.
- `#define DIO_WRITECHANNELGROUP_ID`

API service ID for `Dio_WriteChannel()` Group function.

- #define `DIO_GETVERSIONINFO_ID`

API service ID for `DIO Get Version()` Info function.

- #define `DIO_MASKEDWRITEPORT_ID`

API service ID for `Dio_MaskedWritePort()` function.

- #define `DIO_INSTANCE_ID`

Instance ID of the Dio driver.

Function Reference

- Dio_LevelType `Dio_ReadChannel` (Dio_ChannelType ChannelId)
Returns the value of the specified DIO channel.
- void `Dio_WriteChannel` (Dio_ChannelType ChannelId, Dio_LevelType Level)
Sets the level of a channel.

6.1.2 Macro Definition Documentation

6.1.2.1 DIO_E_PARAM_CONFIG

```
#define DIO_E_PARAM_CONFIG
```

The DIO module is not properly configured.

Definition at line 107 of file `Dio.h`.

6.1.2.2 DIO_E_PARAM_INVALID_CHANNEL_ID

```
#define DIO_E_PARAM_INVALID_CHANNEL_ID
```

Invalid channel name requested.

Definition at line 114 of file `Dio.h`.

6.1.2.3 DIO_E_PARAM_INVALID_PORT_ID

```
#define DIO_E_PARAM_INVALID_PORT_ID
```

Invalid port name requested.

Definition at line 121 of file `Dio.h`.

6.1.2.4 DIO_E_PARAM_INVALID_GROUP_ID

```
#define DIO_E_PARAM_INVALID_GROUP_ID
```

Invalid ChannelGroup id passed.

Definition at line 128 of file `Dio.h`.

6.1.2.5 DIO_E_PARAM_POINTER

```
#define DIO_E_PARAM_POINTER
```

API service called with a NULL pointer.

In case of this error, the API service shall return immediately without any further action, beside reporting this development error.

Definition at line 138 of file [Dio.h](#).

6.1.2.6 DIO_E_PARAM_LEVEL

```
#define DIO_E_PARAM_LEVEL
```

API service called with invalid channel level value.

In case of this error, the API service shall return immediately without any further action, beside reporting this development error.

Definition at line 148 of file [Dio.h](#).

6.1.2.7 DIO_READCHANNEL_ID

```
#define DIO_READCHANNEL_ID
```

API service ID for [Dio_ReadChannel\(\)](#) function.

Parameters used when raising an error/exception.

Definition at line 155 of file [Dio.h](#).

6.1.2.8 DIO_WRITECHANNEL_ID

```
#define DIO_WRITECHANNEL_ID
```

API service ID for [Dio_WriteChannel\(\)](#) function.

Parameters used when raising an error/exception.

Definition at line 161 of file [Dio.h](#).

6.1.2.9 DIO_FLIPCHANNEL_ID

```
#define DIO_FLIPCHANNEL_ID
```

API service ID for [Dio_FlipChannel\(\)](#) function.

Parameters used when raising an error/exception.

Definition at line 167 of file [Dio.h](#).

6.1.2.10 DIO_READPORT_ID

```
#define DIO_READPORT_ID
```

API service ID for `Dio_ReadPort()` function.

Parameters used when raising an error/exception.

Definition at line 173 of file [Dio.h](#).

6.1.2.11 DIO_WRITEPORT_ID

```
#define DIO_WRITEPORT_ID
```

API service ID for `Dio_WritePort()` function.

Parameters used when raising an error/exception.

Definition at line 179 of file [Dio.h](#).

6.1.2.12 DIO_READCHANNELGROUP_ID

```
#define DIO_READCHANNELGROUP_ID
```

API service ID for `Dio_ReadChannel()` Group function.

Parameters used when raising an error/exception.

Definition at line 185 of file [Dio.h](#).

6.1.2.13 DIO_WRITECHANNELGROUP_ID

```
#define DIO_WRITECHANNELGROUP_ID
```

API service ID for `Dio_WriteChannel()` Group function.

Parameters used when raising an error/exception.

Definition at line 191 of file [Dio.h](#).

6.1.2.14 DIO_GETVERSIONINFO_ID

```
#define DIO_GETVERSIONINFO_ID
```

API service ID for `DIO Get Version()` Info function.

Parameters used when raising an error/exception.

Definition at line 198 of file [Dio.h](#).

6.1.2.15 DIO_MASKEDWRITEPORT_ID

```
#define DIO_MASKEDWRITEPORT_ID
```

API service ID for `Dio_MaskedWritePort()` function.

Parameters used when raising an error/exception.

Definition at line 204 of file [Dio.h](#).

6.1.2.16 DIO_INSTANCE_ID

```
#define DIO_INSTANCE_ID
```

Instance ID of the Dio driver.

Definition at line 212 of file [Dio.h](#).

6.1.3 Function Reference

6.1.3.1 Dio_ReadChannel()

```
Dio_LevelType Dio_ReadChannel (
    Dio_ChannelType ChannelId )
```

Returns the value of the specified DIO channel.

This function returns the value of the specified DIO channel.

Parameters

in	<i>ChannelId</i>	Specifies the required channel id.
----	------------------	------------------------------------

Returns

Returns the level of the corresponding pin STD_HIGH or STD_LOW.

6.1.3.2 Dio_WriteChannel()

```
void Dio_WriteChannel (
    Dio_ChannelType ChannelId,
    Dio_LevelType Level )
```

Sets the level of a channel.

If the specified channel is configured as an output channel, this function shall set the specified level on the specified channel. If the specified channel is configured as an input channel, this function shall have no influence on the physical output and on the result of the next read service.

Parameters

in	<i>ChannelId</i>	Specifies the required channel id.
in	<i>Level</i>	Specifies the channel desired level.

6.2 Dio IPL

6.2.1 Detailed Description

Types Reference

- typedef uint32 [Gpio_Dio_Ip_PinsChannelType](#)
Type of a GPIO channel representation Implements : Gpio_Dio_Ip_PinsChannelType_Class.
- typedef uint8 [Gpio_Dio_Ip_PinsLevelType](#)
Type of a port levels representation. Implements : Gpio_Dio_Ip_PinsLevelType_Class.

Function Reference

- void [Gpio_Dio_Ip_WritePin](#) (GPIO_Type *const base, [Gpio_Dio_Ip_PinsChannelType](#) pin, [Gpio_Dio_Ip_PinsLevelType](#) value)
Write a pin of a port with a given value.
- void [Gpio_Dio_Ip_WritePins](#) (GPIO_Type *const base, [Gpio_Dio_Ip_PinsChannelType](#) pins)
Write all pins of a port.
- [Gpio_Dio_Ip_PinsChannelType](#) [Gpio_Dio_Ip_GetPinsOutput](#) (const GPIO_Type *const base)
Get the current output from a port.
- void [Gpio_Dio_Ip_SetPins](#) (GPIO_Type *const base, [Gpio_Dio_Ip_PinsChannelType](#) pins)
Write pins with 'Set' value.
- void [Gpio_Dio_Ip_ClearPins](#) (GPIO_Type *const base, [Gpio_Dio_Ip_PinsChannelType](#) pins)
Write pins to 'Clear' value.
- void [Gpio_Dio_Ip_TogglePins](#) (GPIO_Type *const base, [Gpio_Dio_Ip_PinsChannelType](#) pins)
Toggle pins value.
- [Gpio_Dio_Ip_PinsChannelType](#) [Gpio_Dio_Ip_ReadPins](#) (const GPIO_Type *const base)
Read input pins.
- [Gpio_Dio_Ip_PinsLevelType](#) [Gpio_Dio_Ip_ReadPin](#) (const GPIO_Type *const base, [Gpio_Dio_Ip_PinsChannelType](#) pin)
Read input pin.

6.2.2 Types Reference

6.2.2.1 Gpio_Dio_Ip_PinsChannelType

typedef uint32 [Gpio_Dio_Ip_PinsChannelType](#)

Type of a GPIO channel representation Implements : [Gpio_Dio_Ip_PinsChannelType_Class](#).

Definition at line 120 of file [Gpio_Dio_Ip.h](#).

6.2.2.2 Gpio_Dio_Ip_PinsLevelType

typedef uint8 [Gpio_Dio_Ip_PinsLevelType](#)

Type of a port levels representation. Implements : [Gpio_Dio_Ip_PinsLevelType_Class](#).

Definition at line 126 of file [Gpio_Dio_Ip.h](#).

6.2.3 Function Reference

6.2.3.1 Gpio_Dio_Ip_WritePin()

```
void Gpio_Dio_Ip_WritePin (
    GPIO_Type *const base,
    Gpio\_Dio\_Ip\_PinsChannelType pin,
    Gpio\_Dio\_Ip\_PinsLevelType value )
```

Write a pin of a port with a given value.

This function writes the given pin from a port, with the given value ('0' represents LOW, '1' represents HIGH).

Parameters

<i>base</i>	GPIO base pointer (PTA, PTB, PTC, etc.)
<i>pin</i>	pin number to be written
<i>value</i>	pin value to be written • 0: corresponding pin is set to LOW • 1: corresponding pin is set to HIGH

6.2.3.2 Gpio_Dio_Ip_WritePins()

```
void Gpio_Dio_Ip_WritePins (
    GPIO_Type *const base,
    Gpio\_Dio\_Ip\_PinsChannelType pins )
```

Write all pins of a port.

This function writes all pins configured as output with the values given in the parameter pins. '0' represents LOW, '1' represents HIGH.

Parameters

in	<i>base</i>	GPIO base pointer (PTA, PTB, PTC, etc.)
in	<i>pins</i>	Pin mask to be written • 0: corresponding pin is set to LOW • 1: corresponding pin is set to HIGH

6.2.3.3 Gpio_Dio_Ip_GetPinsOutput()

```
Gpio_Dio_Ip_PinsChannelType Gpio_Dio_Ip_GetPinsOutput (
    const GPIO_Type *const base )
```

Get the current output from a port.

This function returns the current output that is written to a port. Only pins that are configured as output will have meaningful values.

Parameters

in	<i>base</i>	GPIO base pointer (PTA, PTB, PTC, etc.)
----	-------------	---

Returns

GPIO outputs. Each bit represents one pin (LSB is pin 0, MSB is pin 31). For each bit:

- 0: corresponding pin is set to LOW
- 1: corresponding pin is set to HIGH

6.2.3.4 Gpio_Dio_Ip_SetPins()

```
void Gpio_Dio_Ip_SetPins (
    GPIO_Type *const base,
    Gpio_Dio_Ip_PinsChannelType pins )
```

Write pins with 'Set' value.

This function configures output pins listed in parameter pins (bits that are '1') to have a value of 'set' (HIGH). Pins corresponding to '0' will be unaffected.

Parameters

in	<i>base</i>	GPIO base pointer (PTA, PTB, PTC, etc.)
in	<i>pins</i>	Pin mask of bits to be set. Each bit represents one pin (LSB is pin 0, MSB is pin 31). For each bit: • 0: corresponding pin is unaffected • 1: corresponding pin is set to HIGH

6.2.3.5 Gpio_Dio_Ip_ClearPins()

```
void Gpio_Dio_Ip_ClearPins (
    GPIO_Type *const base,
    Gpio_Dio_Ip_PinsChannelType pins )
```

Write pins to 'Clear' value.

This function configures output pins listed in parameter pins (bits that are '1') to have a 'cleared' value (LOW). Pins corresponding to '0' will be unaffected.

Parameters

in	<i>base</i>	GPIO base pointer (PTA, PTB, PTC, etc.)
----	-------------	---

Parameters

in	<i>pins</i>	Pin mask of bits to be cleared. Each bit represents one pin (LSB is pin 0, MSB is pin 31). For each bit: 0: corresponding pin is unaffected 1: corresponding pin is cleared(set to LOW)
----	-------------	--

6.2.3.6 Gpio_Dio_Ip_TogglePins()

```
void Gpio_Dio_Ip_TogglePins (
    GPIO_Type *const base,
    Gpio_Dio_Ip_PinsChannelType pins )
```

Toggle pins value.

This function toggles output pins listed in parameter pins (bits that are '1'). Pins corresponding to '0' will be unaffected.

Parameters

in	<i>base</i>	GPIO base pointer (PTA, PTB, PTC, etc.)
in	<i>pins</i>	Pin mask of bits to be toggled. Each bit represents one pin (LSB is pin 0, MSB is pin 31). For each bit: 0: corresponding pin is unaffected 1: corresponding pin is toggled

6.2.3.7 Gpio_Dio_Ip_ReadPins()

```
Gpio_Dio_Ip_PinsChannelType Gpio_Dio_Ip_ReadPins (
    const GPIO_Type *const base )
```

Read input pins.

This function returns the current input values from a port. Only pins configured as input will have meaningful values.

Parameters

in	<i>base</i>	GPIO base pointer (PTA, PTB, PTC, etc.)
----	-------------	---

Returns

GPIO inputs. Each bit represents one pin (LSB is pin 0, MSB is pin 31). For each bit:

- 0: corresponding pin is read as LOW
- 1: corresponding pin is read as HIGH

6.2.3.8 Gpio_Dio_Ip_ReadPin()

```
Gpio_Dio_Ip_PinsLevelType Gpio_Dio_Ip_ReadPin (
    const GPIO_Type *const base,
    Gpio_Dio_Ip_PinsChannelType pin )
```

Read input pin.

This function returns the current input value of the given pin from port. Only pin configured as input will have meaningful value.

Parameters

in	<i>base</i>	GPIO base pointer (PTA, PTB, PTC, etc.)
in	<i>pin</i>	Pin index (0,1,2,3,...,15)

Returns

GPIO input value for coressponding pin

- 0: corresponding pin is read as LOW
- 1: corresponding pin is read as HIGH

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.